

Design Document: Introduction to Game Programming

Working-Title: Thrust-Clone

Arsenii Parhom; 2606295
Irmak Damla Özdemir; 2606395
Max Tremel; 2606396

Feature 2: Game State, Menu, Textures & Level Design

What it does

Provides the game's structure: the flow from the main menu through the levels to game over, the scoring board, the playable levels themselves, and the visual appearance. Without it, the other features have no context in which to run.

Architecture

At the top is the `GameStateManager` implemented as a state machine with the states [Menu, Play, Pause, LevelDone, GameOver, ScoringBoard]. It defines allowed transitions and raises events when the state changes. Levels are built as tilemaps with collision shapes per tile. A LevelLoader will read level data and spawns objects (rocket, cargo, fuel, guns) from marker tiles or an object layer. This makes levels easy to create and small in storage. As textures, open license resources will be used.

Dependencies / Risks

Dependencies: Most features engage with the game state, like collisions (the Failed state has to remove a live), landing (as the success of one level). They all emit events and the methods in the `GameStateManager` decide what to do.
Risk: Integration conflicts, since all features come together here. An early agreement on events and interfaces reduces this.
Risk: Cave passages that are too narrow for the pendulum physics which make the game to hard. Levels have to be tested with the finished pod mechanic.
Risk: Level design can take much more time as planned.
Risk: Get only assets that are allowed to use.

Acceptance criteria

Minimum (playable):

The game starts in the main menu with Start and Quit. The game can be paused and resumed. Death leads to a game over screen with a restart option. Delivering the cargo leads to a level complete screen. One handcrafted level with working wall collision and all objects placed.

Standard:

The level loader builds levels from data and spawns all objects automatically. 3 levels with increasing difficulty, loaded in sequence. Slope tiles with matching collision shapes. Consistent textures for the rocket, walls, objects, and background.

Score board where the best Score are permanently safed. The score could be the best time or left fuell.

Nice to have:

Procedurally generated caves using cellular automata, with a flood-fill check that the level is flyable and the cargo is reachable. Automatic tile selection at edges (auto-tiling). Level select and a settings menu (e.g. volume, contrast, switched inputs). Additional obsticals in later levels, like better guns, thrust problems, inverted gravity, fuell leakage, usw....